PROJECT CODE MAP · 10 NOV 2026

A map of the whole SOFC POC folder

What the Understand Anything tool found in the project folder, written for readers who do not write code. Interactive version: https://claude.ai/artifact/FC7CWR2x8Sdi6dD2MCAdAB

Contents: 1 Overview · 2 How the parts connect · 3 The 14-step tour · 4 How the map was made and how to use it

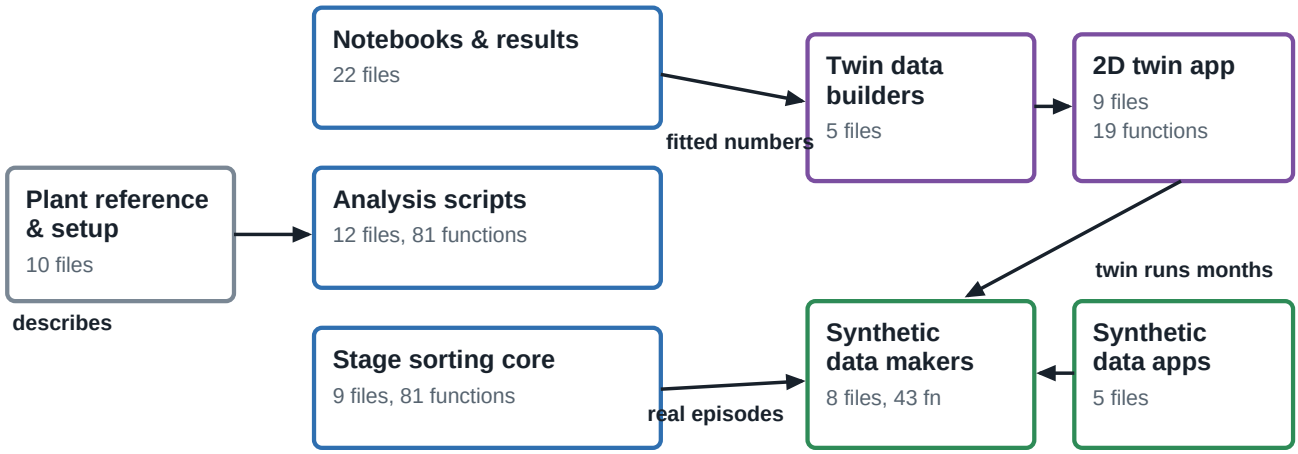
1 / 4 · OVERVIEW

Eight areas, in the order the work flows

A tool called **Understand Anything** read every project file in the POC folder and drew a map: what each file does, and which files depend on which. It sorted the project into **8 areas** and wrote a **14-step reading tour**.

Why it matters: a newcomer can find their way without reading 80 files. The map also shows which files many others depend on, so you can see what a change might break before you make it.

Files read	Parts on the map	Links	Areas	Tour steps
80	308	692	8	14



Colour = family: grey reference, blue analysis, purple digital twin, green synthetic data. An arrow means "hands something to". Source: .ua/knowledge-graph.json.

Area	What it holds	Files	Functions *
Plant reference & setup	Stage definitions, sensor-to-device map, literature notes, start-here README	10	0
Stage sorting core	Labels every minute of plant data with a stage and finds trips (emergency stops)	9	81
Analysis scripts	One script per question: heaters, warming speed, availability, efficiency, 60 kW stretch, twin numbers	12	81
Notebooks & results	16 notebooks (one per question) and their saved results	22	0
Synthetic data makers	Episode library, stage-pattern maker, episode filler, twin dataset runner	8	43
Synthetic data apps	The two point-and-click apps that drive the makers	5	2
Twin data builders	Tools that bake analysis results into the twin's data files	5	1
2D twin app	Clickable plant drawing, stack heat physics, buttons, offline install	9	19

Area	What it holds	Files	Functions *
Total		**80**	**227**

** A function is a named piece of code that does one job. Notebooks are mapped as whole files, so they show 0.*

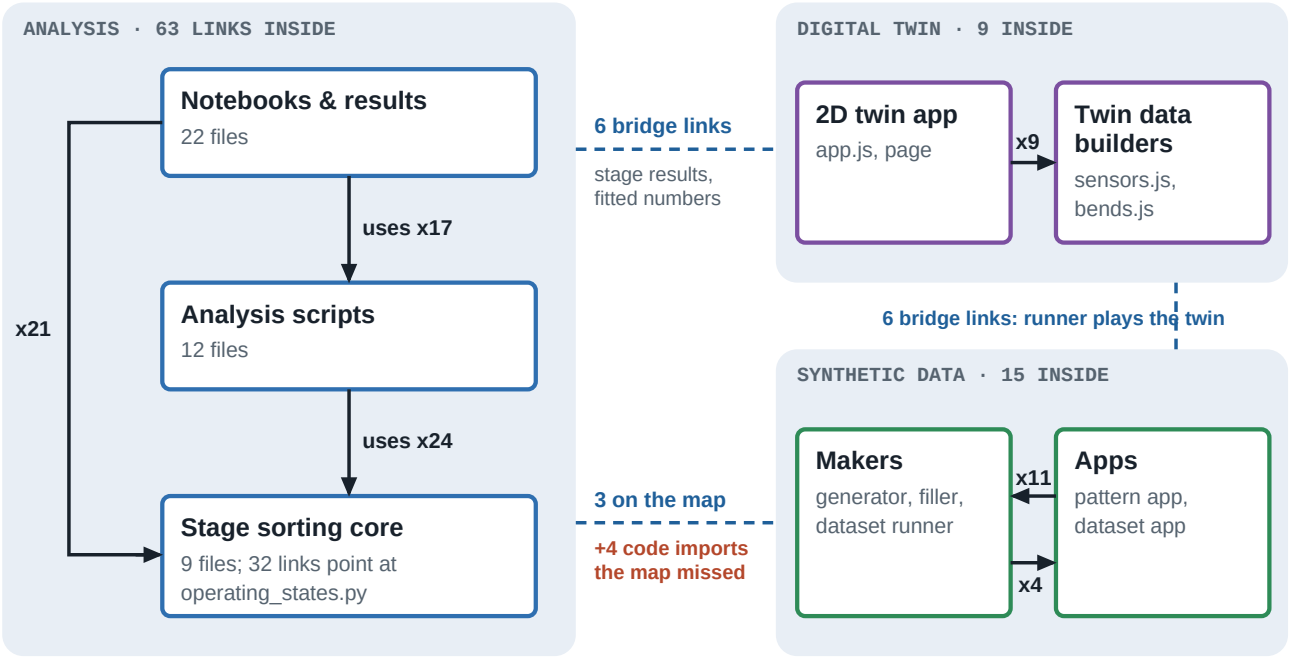
IF YOU REMEMBER ONE THING

The POC is now one map: 80 files in 8 areas, laid out in the order the work flows, from plant reference to analysis to synthetic data and the twin.

Three islands joined by a few bridges

The project is three groups of code that mostly keep to themselves: **analysis**, **synthetic data** and the **digital twin**. The map counts only 15 links between them. Most are a file one group writes and another reads, or a program that runs another. One kind is undercounted: synthetic-data code that borrows analysis code directly.

Why it matters: you can change code inside one island with little risk to the others. The risky spots are the bridges and the few keystone files many others depend on.

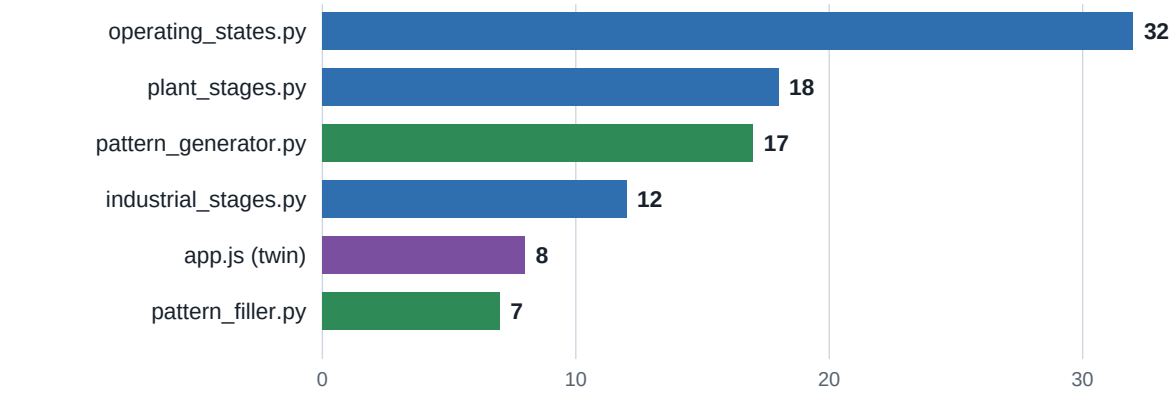


Solid arrow = one area uses another, with the number of links. Dashed blue = a bridge between islands. Red = links found by hand that the map does not hold. Not drawn: 20 "describes" links from reference documents.

What the map misses. Four Python files outside 03_data_analysis borrow its code by adding `03_data_analysis/scripts` to the folders Python searches. The import scanner caught 3 of these 7 borrowings and missed 4: `pattern_generator.py` → `operating_states.py` and `flow_check.py`; `pattern_filler.py` → `operating_states.py` and `plant_stages.py`.

The keystone files

Links pointing at each file (as the map holds them, so slightly low for the analysis files).



Bar colour = island. Source: .ua/knowledge-graph.json.

Keystone	What it does	Before you change it
`operating_states.py`	Labels every minute with a stage and a gas/air phase	Re-run the rule tests and one notebook per stage
`plant_stages.py`	Turns those labels into the 8 decided plant stages	Twin tools read its outputs; table maker and filler import it
`pattern_generator.py`	Draws a realistic random sequence of stages	Pattern app and filler call it; it imports the stage core
`app.js`	The twin's physics, stage rules and buttons	The dataset runner plays this same file

IF YOU REMEMBER ONE THING

Change code inside one island with little worry, but take care with operating_states.py (32+ links) and with anything that crosses between islands.

The 14-step tour, from the plant to the twin

The map comes with a reading path: what the plant is, how data is sorted into stages, the questions answered, then the synthetic data and the twin built from those answers.

Why it matters: this is the order a new team member should read the project in. The same tour is clickable in the dashboard.

1. Project overview
2. The plant reference
3. Cleaning the logger data
4. Stage sorting engine
5. Trips and industry stages
6. The eight plant stages
7. One notebook per question
8. Warming speed, twin physics
9. Plant performance
10. Synthetic data from episodes
11. Building the twin's data
12. The 2D digital twin
13. Running the twin offline
14. Months of data from the twin

Dot colour = family. Source: the tour section of .ua/knowledge-graph.json.

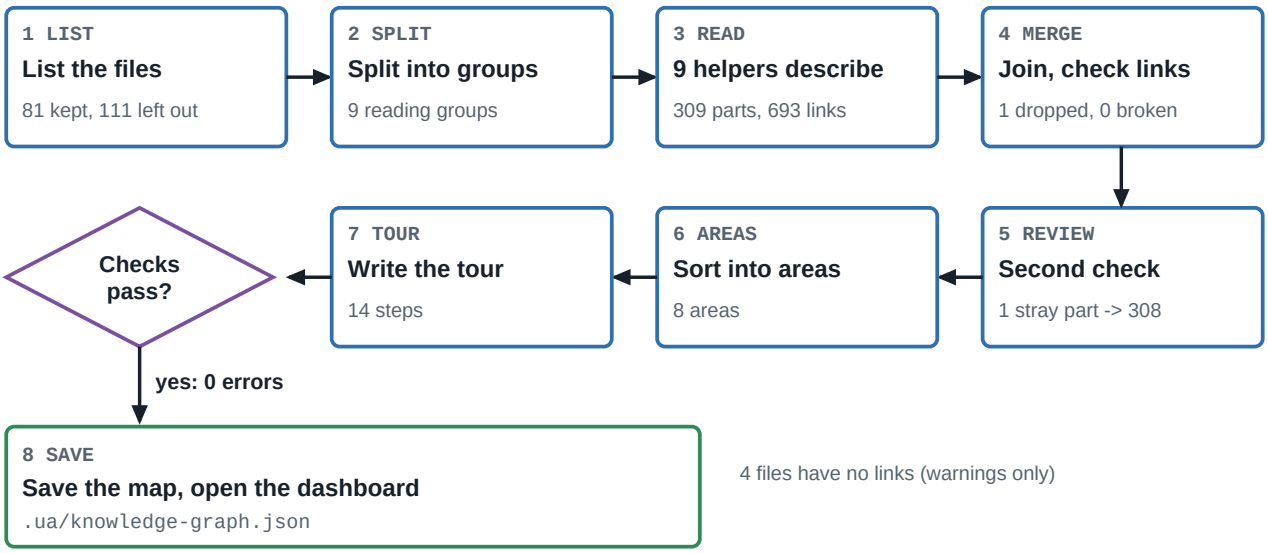
Step	Where to look
1-2 Learn the plant	README.md, 01_docs/operational-modes.md, component_sensor_map_en.md
3-6 Sort the data	scripts/downsample.py, operating_states.py, trip_check.py, industrial_stages.py, plant_stages.py, tests/test_rules.py
7-9 Answer questions	03_data_analysis/README.md and notebooks, slopes_jerk.py, twin_fits.py, availability.py, efficiency_check.py, rated_power_extrapolation.py
10 Make new data	04_synthetic_data: build_library.py, pattern_generator.py, pattern_filler.py, pattern_app.py
11-13 Build the twin	05_twin_2d/tools/prep.py, bends.py, app.js, index.html, app.bat, sw.js
14 Close the loop	04_synthetic_data/twin_dataset.js, dataset_app/, make_tables.py

IF YOU REMEMBER ONE THING

Read the project in this order: plant, stages, questions, synthetic data, twin. Step 4, the stage sorting engine, is the file everything else stands on.

How the map was made, and how to use it

The tool works like a small survey team: it lists the files, hands them out in groups to AI helpers (each given one small job), joins their notes into one map, checks it twice, sorts it into areas and writes the tour.



The diamond is the final quality gate: every link must point at a real part and every file must sit in exactly one area.

Left out on purpose

Left out	Why
`02_data/` (28 GB of plant logs)	Data, not logic
`06_deliverables/`, `graphify-out/`, `04_synthetic_data/datasets/`	Finished reports and generated outputs
Images, CSV, parquet, Excel, PDF	Results, not code (same rules as your graphify setup)

How to use the map (type these in Claude Code)

To	Type	What happens
See it	`/understand-dashboard`	Opens the interactive map; click any part to see what it does and links to
Ask it	`/understand-chat your question`	Answers plain questions using the map
Keep it current	`/understand`	Re-reads only the files that changed since the last run

45 files were reformatted while the map was being made. At 14:00, something outside this run re-laid out the Python scripts, notebooks and twin JavaScript. It sorted imports, wrapped long lines, and in a few files removed unused imports. Notebook results are unchanged. It was most likely another Claude session open at the same time. It was not undone. Check git diff before the next commit.

Also: 4 of 7 cross-folder imports are missing (page 2); notebooks are mapped as whole files; line numbers may be slightly off for reformatted files; `01_docs/SKILL.md` is an unrelated page-design guide.

IF YOU REMEMBER ONE THING

The map is saved in the .ua folder. Open it with /understand-dashboard, and re-run /understand after changes to keep it current.